

JEUS

JNDI Guide

Version 3.0

We Speak in Technology

 ***Tmax Soft***

Part Number : K:JSJN-1015-01-300

Last Revised : 15, October, 2001

Print :

Copyright Information

Copyright©2001 Tmax Soft Co., Ltd. All Rights Reserved.

Tmax Soft Co., Ltd

18F Glass Tower, 946-1, Daechi-Dong, Kangnam-Gu

Seoul, Korea 135-708

This software and documents are made available only under the terms of the Tmax Soft License Agreement and may be used or copied only in accordance with the terms of this agreement.

No part of this document may be reproduced, transmitted, or translated in any form or by any means, electronic, mechanical, manual, or optical, without the prior written permission of

Tmax Soft Co., Ltd.

Trademark

Tmax, WebtoB, WebT, and JEUS are registered trademarks of Tmax Soft Co., Ltd.

서 문

이 책은 JEUS 3.0의 JNDI에 대한 설명을 하고 있다. 이 책은 JEUS JNDI의 구조와 이를 이용한 개발에 대한 내용을 담고 있다.

이 책의 구성

이 책은 단계적으로 JNDI의 개념에 대한 기본적인 내용과 JEUS JNDI의 세부적인 부분, 또한 JEUS JNDI를 사용하고 JEUS Naming Server를 설정하는 데에 대한 설명을 담고 있다. 각각의 부분은 단계적으로 해당하는 내용에 대한 설명을 하고 있으며 각 부분 사이에 상호 보완적인 연결을 가지고 있다. 다음은 이 책을 구성하고 있는 각 장에 대한 설명이다.

■ I 장, JEUS JNDI에 대하여

이 장에서는 기본적인 JNDI에 대한 설명과 JEUS JNDI에 대한 설명을 하고 있다. 개념적인 설명과 함께 구조에 대한 설명을 다루고 있다.

■ II 장, JEUS Naming Service 이용하기

이 장에서는 JEUS에서 제공하고 있는 Name Service를 이용하는 방법과 클러스터링 환경에서 JEUS JNDI를 사용하는 데에 대한 설명을 하고 있다. JEUS JNDI가 가지고 있는 특성과 함께 제공하고 있는 기능들에 대한 설명을 다루고 있다.

■ III 장, JEUS Naming Server 설정하기

이 장에서는 JEUS Naming Server를 동작시키고 클러스터링 환경에서의 설정을 하는 사항을 설명하고 있다.

■ IV 장, JEUS Directory Service 이용하기

이 장에서는 JEUS에서 제공하고 있는 Directory Service를 이용하는 방법을 설명하고 있다.

■ V 장, 환경 변수 목록

이 장에서는 JEUS JNDI를 설정할 수 있는 환경 변수에 대한 목록을 정리하고 있다.

누가 이 책을 읽어야 하는가

이 책은 JEUS JNDI를 이용해 엔터프라이즈 환경에서 응용프로그램을 개발하는 사람과 JEUS 서버를 관리하는 사람에게 필요한 정보를 담고 있다. 다음은 각 장에서 다루고 있는 내용이 필요한 사람들을 나타낸 것이다.

- **JEUS 관리자**

I장 JEUS JNDI에 대하여, III장 Naming Server 설정하기, V장 환경 변수 목록

- **엔터프라이즈 응용프로그램 개발자**

I장 JEUS JNDI에 대하여, II장 JEUS JNDI 사용하기, V장 환경 변수 목록

관련된 책들

JEUS에 관한 매뉴얼은 다음과 같은 것들이 있다.

- **JEUS Installation Guide**

JEUS 시스템의 설치를 위한 준비 작업과 설치 요령, 설치 후의 처리 사항에 대해 설명하고 있는 매뉴얼이다.

- **JEUS Getting Started**

JEUS를 처음 설치한 후, EJB/Servlet Engine을 설정과 Example 빈들과, Servlet/JSP를 실행하는데 필요한 설명이다..

- **JEUS Administration Guide**

JEUS를 어떻게 관리 하는지에 관하여 설명하고 있다. JEUS의 전반적인 구조에 대해서 자세히 설명하며, 실제 JEUS 시스템을 구성하는 방법에 대해서 예를 들어가면서 설명한다. 이 책은 JEUS WebContainer Guide 및 JEUS Enterprise JavaBeans Guide 매뉴얼과 더불어 시스템 관리자가 꼭 읽어야 하는 책이다.

- **JEUS Administration Tool Guide**

JEUS 전체를 관리하고, 모니터링하며, JEUS 시스템에 관련된 configuration 파일의 생성을 도와주는 Administration Tool에 관한 내용을 다루고 있는 매뉴얼이다.

- **JEUS Deployment Tool Guide**

어플리케이션 프로그램 개발자가 작성된 EJB 빈들로 EJB 모듈을 구성하거나 Servlet, JSP 프로그램과 함께 전체 시스템을 구성할 때 이들을 하나의 모듈로 packing하여 주고, packing 된것을 시스템에 deploy하는 jdeploy Tool에 관한 내용을 다루고 있는 매뉴얼이다.

- **JEUS Client Application Development Guide**

JEUS 서비스를 클라이언트가 사용할 수 있도록 기본 환경을 제공하는 Client Container의 구조와 사용 방법을 설명한다. Client Container는 기본적으로 JEUS의 네이밍 서비스, Scheduler, Security를 이용할 수 있는 환경을 제공한다.

- **JEUS Enterprise JavaBeans Guide**

엔터프라이즈 자바 빈즈에 대한 설명과 JEUS의 EJB의 구조와 운영, 개발 등에 필요한 내용을 다루고 있는 매뉴얼이다.

- **JEUS JDBC Guide**

JDBC에 대한 기본적인 개념과 DB Vendor별 Sample Programming을 기술하고 있다.

- **JEUS JMS Guide**

Java Messaging Service에 관한 설명과 JEUS JMS를 사용하고 설정하는 방법을 기술하고 있다.

- **JEUS JTS Guide**

Jeus 에서 제공되고 있는 Transaction 서비스의 이용 방법과 관련 설정 및 관리 방법을 설명하고 있는 매뉴얼이다.

- **JEUS Scheduler Guide**

일정 간격으로 발생하는 이벤트를 처리 할 수 있는 JEUS Scheduler의 기본적인 개념과 간단한 예제를 제시하고 있다.

- **JEUS Security Guide**

Security에 대한 일반적인 설명과 JEUS Security를 사용하고 설정하는 방법을 기술하고 있다.

- **JEUS Web Container Guide**

JEUS Web Container의 관리와 이를 이용한 servlet, JSP 개발에 대한 내용을 담고 있다. Web Container 관리에 대해서는 모든 내용을 빠짐없이 기술하였고, 개발 측면에 대해서는 Servlet과 JSP의 기본적인 요소에 대한 설명과 각각의 구현 요소에 대해 간략히 설명하였다.

■ JEUS WebtoB Light Guide

JEUS에 내장된 WebServer인 WebtoB Light의 기능을 적절히 활용할 수 있도록 하기 위한 목적으로 만들어졌다. WebtoB Light의 전반적인 구조, 환경파일 작성 방법 및 WebServer의 관리 방법을 기술하고 있다.

■ JEUS XML Guide

XML의 개념과 JEUS에서 XML관련 Application을 사용하는 방법을 설명 하고 있다.

■ JEUS CAS Guide

CAS(Client Access Service)의 개념에 대한 기본적인 내용과 JEUS CAS의 세부적인 부분, 또한 JEUS CAS를 설정하고 사용하는 데에 대한 설명을 담고 있다.

글씨체에 대해

다른 대부분의 책과 같이 java의 클래스 이름이나 소스 코드에서는 **Courier New** 글씨체를 사용하였다. 또한 주의할 사항은 **Note** : 와 같이 표시해 두었다.

JEUS 제품별 기능

Edition	기 능
Base	WebToB Light, Servlet, JSP, jadmin, JDBC Connection Pool, Jeus Manager, Security
Standard	JEUS Base, EJB, JTS(local transaction only), JMS, jdeploy, JNDI
Enterprise	JEUS standard, JTA(multi node, 이기종 DB)

차 례

서 문.....	2
이 책의 구성	2
누가 이 책을 읽어야 하는가	3
관련된 책들.....	3
글씨체에 대해.....	5
JEUS 제품별 기능	5
I. JEUS JNDI에 대하여	8
1. 개요.....	8
2. JNDI란 무엇인가.....	8
3. JEUS JNDI 개요	8
4. JEUS NAMING SERVER의 구조	9
4.1 JNSServer.....	10
4.2 JNSLocal.....	11
4.3 클러스터링	12
II. JEUS NAME SERVICE 이용하기.....	15
1. 개요.....	15
2. JAVA CLIENT에서 JEUS JNDI 사용하기	15
2.1 JEUS 환경 설정	16
2.2 Initial Context 환경 변수 설정하기.....	16
2.3 Context에서 Named Object 찾기	19
2.4 Named Object에서 Object Reference 얻기	20
2.5 Context 종료하기.....	20
3 서버 측 객체에서 JEUS JNDI 사용하기.....	20
3.1 InitialContext 생성하기.....	21
3.2 Context에서 Named Object 찾기	22
3.3 Named Object에서 Object Reference 얻기	22
3.4 Context 종료하기.....	22
4. 클러스터링 환경에서 JEUS JNDI 사용하기.....	23
4.1 클러스터링 환경 구성하기	23
4.2 클러스터링 환경에 객체 등록하기	24
III. JEUS NAMING SERVER 설정하기.....	28

1. 개요.....	28
2. 기본 설정 사항.....	28
2.1 JNSServer 설정.....	28
2.2 JNSLocal 설정 (Server Side).....	30
2.3 JNSLocal 설정 (Client Side).....	32
3. 클러스터링 환경 설정.....	33
3.1 JNSServer 설정.....	34
3.2 JNSLocal 설정 (Server Side).....	34
3.3 JNSLocal 설정 (Client Side).....	34
4. 보안 설정.....	34
4.1 사용자 인증 설정.....	35
4.2 접근 제어 설정.....	37
IV. JEUS DIRECTORY SERVICE 이용하기.....	41
1. 개요.....	41
2. ATTRIBUTE SERVICE.....	41
2.1 Attribute 등록.....	41
2.2 Query.....	41
2.2 Query – 검색 필터(SearchFilter) 이용하기.....	42
2.2 Query – 검색 제어(SearchControl) 이용하기.....	42
2.2 NamingEnumeration.....	44
3. EVENT SERVICE.....	44
3.1 NamespaceChangeListener.....	44
3.2 ObjectChangeListener.....	45
3.3 Listener 등록하기.....	45
V. 환경 변수 목록.....	46
1. 개요.....	46
2. JEUS JNDI 환경 변수.....	46

I . JEUS JNDI에 대하여

1. 개요

이 장에서는 JNDI에 대한 기본적인 개념과 함께 JEUS에서 제공하고 있는 JNDI의 기능 및 구조에 대하여 설명하겠다.

2. JNDI란 무엇인가

엔터프라이즈 환경에서 응용프로그램은 네트워크 안에 위치한 객체들을 찾기 위해서 Naming Service를 이용하게 된다. Naming Service를 통해서 객체에 이름을 부여하고 원하는 객체를 찾아 사용할 수 있게 되는 것이다.

여기에서 설명하는 Java Naming and Directory Interface (JNDI)는 이러한 Naming Service 와 Directory Service를 Java 응용 프로그램에서 사용할 수 있도록 해 주는 Application Programming Interface (API) 이다. JNDI는 Naming Service, 또는 Directory Service를 제공하는 시스템의 종류에 관계 없이 Java 응용 프로그램에서 공통된 API를 통해서 서비스를 받을 수 있도록 하며, Sun Microsystems Java 2 Platform Enterprise Edition (J2EE) 기술의 중요한 요소이다.

현재까지 JNDI 표준은 1.2.1 버전까지 발표되었으며 다양한 종류의 Naming, Directory Service를 연결하여 사용할 수 있는 환경을 제공한다.

3. JEUS JNDI 개요

JEUS 3.0에서 제공하는 JNDI는 JNDI 1.2.1 API를 준수하고 있으며 Sun Microsystems, Inc에서 정의한 표준 JNDI API의 클래스들을 기반으로 하고 있다. 따라서 JNDI SPI를 준수하는 Service Provider 들의 종류에 상관 없이 엔터프라이즈 환경에 서비스를 제공하게 된다. 또한 JEUS 3.0에서는 JNDI SPI를 준수하여 JEUS Naming Service를 제공하고 있으며 이는 엔터프라이즈 환경에 적합하게 구성되어 있다.

JEUS JNDI에서는 다음과 같은 종류의 기능을 엔터프라이즈 환경에 제공한다.

-
- 클라이언트가 JEUS Naming Service에 접근할 수 있도록 한다.
Servlet, JSP, EJB, Java Application Client 등의 응용프로그램에서 JEUS의 Naming Service에 접근하고자 할 때에 JEUS JNDI를 통하여 JEUS JNDI Tree의 내용을 사용하거나 등록할 수 있게 한다.
 - 객체들을 JEUS Namespace에서 사용할 수 있도록 한다.
JEUS에서 서비스하게 되는 응용프로그램이나 공통적으로 사용하는 객체들을 JEUS Namespace에 등록하고 수정할 수 있는 기능을 제공한다. JEUS Namespace에 구성되어 있는 객체들은 JEUS JNDI를 통하여 역시 찾을 수 있으며 응용프로그램등에서 필요한 객체를 찾아 사용할 수 있게 된다.
 - JEUS Namespace에서 객체들을 찾을 수 있도록 한다.
JEUS Namespace에 구성되어 있는 객체들을 JEUS JNDI를 통하여 찾아올 수 있다. JEUS JNDI에서는 RMI 객체와 EJB 객체 등 엔터프라이즈 환경에서 사용할 수 있는 객체들을 모두 찾아와 사용할 수 있으며 Cache 기능을 통해 효율적인 Naming Service를 가능하도록 한다.
 - 클러스터링 환경에서 JEUS Naming Service를 제공한다.
JEUS JNDI에서는 클러스터링 환경에서 전체가 공유하게 되는 JNDI Tree를 구성할 수 있는 기능을 제공하며, 또한 클러스터링 환경의 일부에서만 사용할 수 있는 옵션을 제공하여 효율적인 엔터프라이즈 환경을 구성할 수 있도록 한다.

4. JEUS Naming Server의 구조

JEUS Naming Server는 JEUS JNDI SPI를 통해서 클라이언트에게 Naming Service를 제공하며 클러스터링 환경에서 공통된 JNDI Tree를 구성하고 관리하며 JNSServer와 JNSLocal의 구조를 통하여 응용 프로그램에서 원하는 종류의 Naming Service를 효율적으로 제공하게 된다.

JEUS Naming Server는 Java로 작성되어 있으므로 Java 응용 프로그램에서 등록하고 찾고자 하는 객체를 완벽하게 지원하며 RMI 객체와 EJB 객체, 데이터베이스 Connection Pool 등 엔터프라이즈 환경에서 사용하게 되는 자원들을 효율적으로 서비스한다.

클러스터링 환경에서 JEUS Naming Server는 각각의 노드를 연결하고 공통된 JNDI Tree를 구성하고 관리하는 기능을 수행하는 JNSServer와 각각

의 노드에서 JNSServer와 상호 작용하며 JNDI SPI로부터의 클라이언트 요청을 처리하게 되는 JNSLocal로 구성되며, 이들의 자세한 설명은 다음과 같다.

4.1 JNSServer

JNSServer는 독립적으로 동작할 수 있는 Naming Server의 구성 요소이며 JNDI Tree의 내용을 관리하고 JNSLocal에 JNDI Tree로 접근할 수 있는 방법을 제공한다. 다음의 그림은 JNSServer와 이에 연결된 JNSLocal들의 관계를 나타낸 것이다.

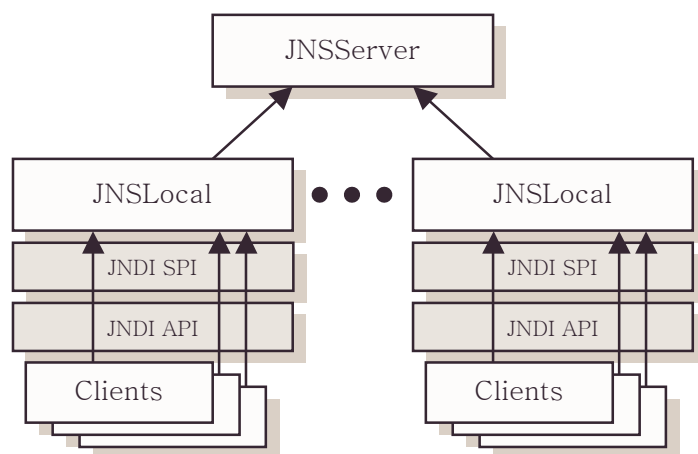


그림 1-1. JNSServer와 JNSLocal의 관계

클러스터링 환경에서는 여러 개의 JNSServer를 연결하여 하나의 공통된 JNDI Tree를 구성할 수 있으며 확장이 가능하고 여러 시에 이를 인지하고 적절한 대응을 할 수 있는 기능을 제공하고 있다. JEUS 3.0에서 JNSServer는 각각의 노드마다 하나씩 실행되어 동작하며 JEUS Manager가 자동으로 실행하여 JNSLocal로부터의 연결을 기다리게 된다.

4.2 JNSLocal

JNSLocal은 JNSServer에 연결되어 클라이언트로부터의 요청을 JNSServer에게 전달하고 JNSServer로부터 돌아온 응답을 클라이언트에게

돌려주는 기본적인 기능을 수행한다. 이와 함께 하나의 JVM에서 동작하는 여러 개의 클라이언트의 요청을 모두 받아들여 JNDI Tree에서 찾아온 객체를 Cache하고 있으면서 클라이언트의 반복적인 요청에 대해서 즉각적인 응답을 줄 수 있는 동작을 한다.

JNSLocal은 JNDI API를 통해서 들어오는 클라이언트의 요청을 처리할 때에 하나의 JVM에 하나씩 실행되어 동작하도록 설계되었으며 따라서 엔터프라이즈 환경에서 EJB나 Servlet등을 실행하기에 적합한 구조를 가지고 있다. JNSLocal이 제공하는 기능을 요약하면 다음과 같다.

■ JNDI Tree로의 다양한 접근 방법 제공

JNSLocal은 JNSServer에게 연결하여 JEUS 환경에서 관리하고 있는 JNDI Tree에 접근하는 방법을 제공한다. 클라이언트의 설정에 따라 객체를 등록하거나 찾아올 때에 전체 JNDI Tree에서 공유할 수 있도록 하거나 일부의 클라이언트들에게서만 사용할 수 있도록 한다.

■ 데이터 Caching 기능 제공

JNSLocal은 클라이언트가 찾아오도록 요청한 객체를 자신의 저장 공간에 Caching하여서 자주 사용되는 객체를 클라이언트들이 쉽게 찾을 수 있도록 한다. 또한 JNS Server와의 통신을 통해서 Caching하고 있는 데이터에 대한 관리를 수행한다.

■ JNSServer와의 Connection 관리 기능 제공

JNSLocal에서는 클라이언트로부터의 요청을 받아들여 하나의 Connection을 이용하여 JNSServer에게 요청을 전달하고 응답을 받아 오게 된다. JNSLocal과 클라이언트 사이에서는 하나의 JVM 안에서 동작하면서 I/O를 일으키지 않고 효율적인 통신을 수행한다.

JNSLocal은 크게 2종류가 있다. JNDI를 이용하는 모듈이 어느곳에 위치해 있는가에 따라 분류된다.

■ Server Side JNSLocal

Engine Container (JVM)당 한 개씩 생겨서 Engine Container의 엔진 내에 존재하는 EJB Bean, Servlet, JSP 등의 모듈등에서 사용할 수 있는 Naming 모듈. 이를 이용하기 위해서는 `java.naming.factory.initial` 을 `jeus.jndi.JNSContextFactory` 로 설정해 주면 되는 데, 이 설정은 Engine Container가 부팅되면서 시스템 프로퍼티로 설정되기 때문에 유저는 이를 이용하기 위해 특별한 세팅을 해 줄 필요가 없다.

■ Client Side JNSLocal

일반 클라이언트, 또는 클라이언트 컨테이너 내의 모듈에서는 Client Side JNSLocal을 이용해야 되는 데, 이를 위해서는 `java.naming.factory.initial` 을 `jeus.jndi.JEUSContextFactory`로 설정하면 된다. Server Side JNSLocal과는 달리 일정시간 통신하는 데이터가 없으면 Connection을 끊고 다음 사용시 다시 Connection을 맺도록 하는 방법으로 리소스를 효율적으로 관리하도록 하고 있다.

4.3 클러스터링

JEUS Naming Server는 클러스터링 환경에서 JNSServer 사이의 연결과 JNSLocal과의 연결에 의해서 공통적인 JNDI Tree를 구성하고, 정보의 변경에 대한 내용을 관리하며 여러 노드의 서비스를 지원하게 된다. 다음 그림은 클러스터링 환경에서의 JNSServer와 JNSLocal 들의 연결을 나타낸 것이다.

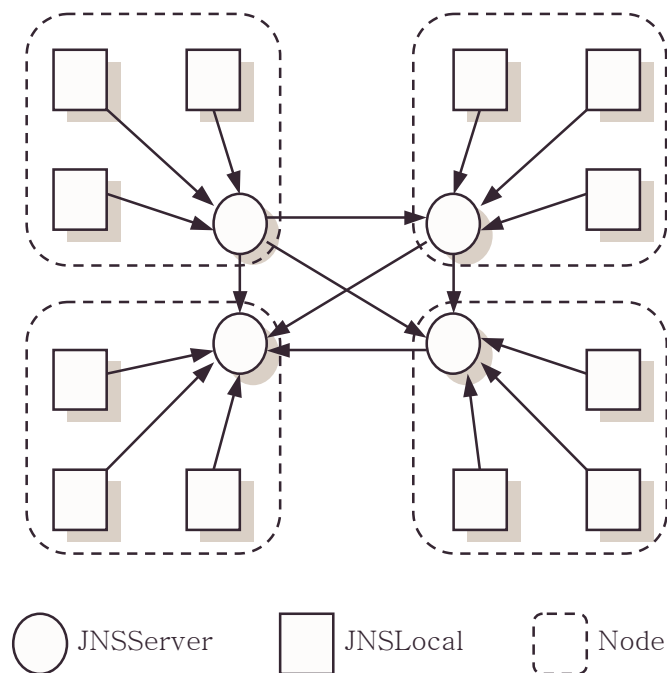


그림 1-2. 클러스터링 환경에서의 구성

JNSServer의 동작은 각 노드에 있는 JEUS Manager가 담당하며 JEUS 시스템이 서비스를 시작할 때에 각각의 JNSServer들이 시작되어 다른 JNSServer들과 연결을 맺게 된다. 한편 각각의 JEUS 엔진들에서는 Naming Service를 받기 위해서 JNDI API의 Initial Context를 얻음과 동시에 JNSLocal을 실행하여 자신의 노드에 있는 JNSServer에게 연결하게 된다.

클러스터링 환경에서 각각의 노드에 있는 JEUS 엔진과 같은 JNDI 클라이언트들이 JNDI Tree의 객체를 찾고자 할 때에는 위와 같은 구조에서 JNSLocal과 자신의 노드에 있는 JNSServer에게 물어봄으로써 원하는 객체를 얻게 된다.

4.3.1 Scalability

앞에서 보인 그림과 같이 이미 구성되어 있는 클러스터에 새로운 노드가 추가될 때에는 JNSServer가 시작하면서 기존의 JNSServer에게 자신의 존재를 알리게 된다. 이 경우 전체의 JNDI Tree를 구성하고 있는 내용에 대해서 각각의 JNSServer들은 자신이 등록하였던 내용들을 새로운 JNSServer에게 전달하여 새로이 추가된 노드에서도 기존의 Namespace에 있던 객체들을 찾을 수 있도록 한다.

이러한 Scalability는 클러스터를 구성하고 있던 노드 중에서 한 JNSServer가 비정상적으로 종료되었다가 다시 시작되었을 때에도 마찬가지로 JEUS Manager에서 새로 시작하는 JNSServer가 다른 JNSServer들에게서 기존의 JNDI Tree의 내용을 얻어올 수 있도록 한다.

II. JEUS Name Service 이용하기

1. 개요

이 장에서는 JEUS JNDI를 사용하여 Java 응용 프로그램을 작성하거나 클러스터링 환경에서 JEUS JNDI를 사용하는 방법에 대한 설명을 하겠다. JNDI API를 통하여 원하는 JEUS Name Service를 제공 받을 수 있도록 설정하는 내용과 클러스터링 환경에서 서비스의 성격에 따라 가질 수 있는 설계 구조에 대한 내용도 함께 포함한다.

2. Java Client에서 JEUS JNDI 사용하기

JEUS JNDI의 SPI에서는 Java Client에게 JEUS로 접근할 수 있도록 InitialContext를 제공한다. 일반적인 InitialContext의 환경 변수를 설정하고, 또한 JEUS JNDI에서 제공하는 환경 변수들을 이용하여 Java Client는 원하는 동작을 수행하여 JEUS JNDI Tree의 내용을 참조하고 서비스를 제공받을 수 있게 된다.

Java Client는 JEUS와 연결을 맺어 동작하기 위해서 다음과 같은 과정을 거쳐야 한다.

- JEUS 환경 설정
- InitialContext 환경 변수 설정
- Context에서 명명된 객체 찾기
- 명명된 객체로부터 Remote 객체의 Reference를 얻어와 기능 수행하기
- 연결을 완료하기

다음에서는 위와 같은 사항들에 대해서 상세하게 설명하고 있으며 클러스터링 환경에서 JEUS JNDI Tree에 객체들을 설정하기 위해서는 본 매뉴얼의 클러스터링 환경에서 JEUS JNDI 사용하기를 참고하기 바란다.

2.1 JEUS 환경 설정

Java Client에서 JEUS Naming Service를 제공받기 위해서는 JEUS 시스템이 설치되어 있어야 한다. JEUS Naming Server는 JEUS가 동작하면서 함께 실행되고, Initial Context를 통해서 JNSLocal을 실행시키게 된다. JEUS JNDI를 사용하기 위해서는 JEUS 클라이언트 모듈을 Java Client와 같은 시스템에 설치하여야 하며, 클라이언트 모듈을 사용하기 위해서 다음과 같이 클래스 패스를 지정하게 된다.

- CLASS_PATH (-classpath)

JEUS 클라이언트 모듈이 설치되어 있는 경로를 %JEUS_CLIENT%라고 할 때에 다음과 같은 클래스 경로를 지정하여 JEUS Naming Service를 받기 위해서 필요한 클래스들을 찾을 수 있도록 해야 한다.

```
-classpath %JEUS_CLIENT%;
```

2.2 Initial Context 환경 변수 설정하기

JEUS JNDI를 이용하여 Java Client가 Naming Service를 받기 위해서는 먼저 JEUS의 Initial Context를 얻기 위한 환경 변수를 설정해야 한다. 다음에서 설명하는 것과 같은 내용을 Initial Context Factory에서 사용할 수 있도록 Hashtable에 구성하여 InitialContext의 생성자에게 인자로 넘겨 주거나 Java 인터프리터의 -D 옵션으로 정의할 수 있다. 우선순위는 두번째 방법이 더 높다.

JEUS에서 제공하는 Initial Context를 생성하기 위해서는 다음과 같은 종류의 환경 변수를 설정할 수 있다. (각 항목 아래에 Java 인터프리터의 -D 옵션으로 정의하는 방법을 보였다.)

- Context.INITIAL_CONTEXT_FACTORY (필수)

JEUS에서 제공하는 Initial Context를 사용하기 위해서는 이 항목을 `jeus.jndi.JEUSContextFactory`로 설정해야 한다. Java 인터프리터의 옵션으로는 다음과 같이 설정할 수 있다.

```
-Djava.naming.factory.initial=jeus.jndi.JEUSContextFactory
```

- Context.URL_PKG_PREFIXES

JEUS Initial Context를 사용하여 URL Scheme을 이용하여 객체를 찾

기 위해서는 이 항목을 `jeus.jndi.jns.url`로 설정해야 한다. Java 인터프리터의 옵션으로는 다음과 같이 설정할 수 있다.

`-Djava.naming.factory.url.pkgs=jeus.jndi.jns.url`

■ Context.PROVIDER_URL

이 환경 변수를 통하여 JEUS JNDI를 사용하기 위해서 JNSLocal이 동작할 때에 JNSServer에게 연결할 수 있는 호스트 주소를 설정할 수 있다. 기본값은 `127.0.0.1`로 설정되어 있으며 원격 호스트의 주소를 설정하게 되면 JNSServer와 JNSLocal을 서로 다른 노드에 분산시킬 수 있다.

`-Djava.naming.provider.url=<hostaddress>`

■ Context.SECURITY_PRINCIPAL

이 환경 변수를 통하여 JEUS Naming Service에서 인증 과정에서 사용하게 되는 사용자의 정보를 설정할 수 있다. 실행되고 있던 스레드에 사용자의 정보가 등록되어 있지 않으면 기본값으로 `guest` 사용자로 설정이 되며 JEUS Security Realm에 등록된 사용자의 이름으로 새로운 사용자의 정보를 사용할 수 있다.

`-Djava.naming.security.principal=<username>`

■ Context.SECURITY_CREDENTIALS

앞에서 설명한 사용자의 이름과 함께 인증 과정을 위해서 사용하는 사용자의 비밀 번호를 설정할 수 있다. 사용자의 이름과 비밀 번호를 통하여 인증에 실패하게 되면 이전에 사용하고 있던 사용자의 정보를 그대로 유지하며, 인증이 성공하게 되면 새로운 사용자의 정보를 사용하게 된다.

`-Djava.naming.security.credentials=<password>`

이와 같은 정보는 Hashtable로 설정하여 Initial Context를 얻는데 사용될 수 있으며, Server-Side Object에서 Initial Context를 얻기 위해서는 기본적인 설정만으로도 JEUS에서 제공하는 Initial Context를 얻을 수 있다. 다음의 코드는 Context.INITIAL_CONTEXT_FACTORY 환경 변수만을 설정하여 JEUS의 Initial Context를 얻는 예를 보여주고 있다.

```
Context ctx = null;
```

```

Hashtable ht = new Hashtable();
ht.put(Context.INITIAL_CONTEXT_FACTORY, "jeus.jndi.JEUSContextFactory");
ht.put(Context.URL_PKG_PREFIXES, "jeus.jndi.jns.url");
ht.put(Context.PROVIDER_URL, "<hostname>");
ht.put(Context.SECURITY_PRINCIPAL, "<username>");
ht.put(Context.SECURITY_CREDENTIALS, "<password>");
try {
    ctx = new InitialContext(ht);
    // successfully got the context
    // use the context
} catch (NamingException ex) {
    // an error occurred
} finally {
    try {
        ctx.close();
    } catch (Exception e) {
        // an error occurred
    }
}

```

위에서 설명한 환경 변수 이외에 JEUS JNDI에서 추가적으로 제공하는 jeus.jndi.JEUSContext에 정의되어 있는 환경 변수를 설정하여 내부적인 JNSLocal의 동작과 클러스터링 환경에서 JNDI Tree를 효율적으로 구성하고 관리할 수 있다. 이에 대한 설명은 본 매뉴얼의 클러스터링 환경에서 JEUS JNDI 사용하기와 3장 JEUS Naming Server 설정하기의 내용을 참고하기 바란다.

2.2.1 Hashtable을 이용하여 설정하기

앞에서 말한 Initial Context를 생성하기 위한 환경 변수들을 설정하기 위해서 Hash Table을 이용할 수 있다. 환경 변수의 값을 설정하기 위해서는 Hash Table에 환경 변수의 이름을 Key 값으로 하고, 환경 변수의 값을 Hash Table의 Entry로 하여 추가한 후에 InitialContext의 생성자에 인자로 전달하면 된다.

JEUS JNDI에서 제공하는 InitialContext를 생성하기 위해서는 앞에서 설명한 것과 같이 `java.naming.factory.initial` 환경 변수의 값을 `jeus.jndi.JEUSContextFactory`로 설정해야 한다.

2.2.2 Security Context 구성하기

현재 동작하고 있는 스레드에 JEUS Security Realm에서 관리하고 있는 사용자의 정보를 설정하고자 할 때에는 JEUS JNDI에서 Initial Context를 생성하는 환경 변수를 사용할 수 있다. 이 때에 사용되는 환경 변수는 사용자의 이름을 지정하는 `java.naming.security.principal`과 비밀번호를 기록할 수 있는 `java.`

`naming.security.credentials`이며 이들의 정보를 가지고 JEUS JNDI Initial Context를 생성하면서 Security Context를 구성하게 된다.

```
Hashtable ht = new Hashtable();
ht.put("java.naming.security.principal", "<username>");
ht.put("java.naming.security.credentials", "<password>");
Context ctx = new InitialContext(ht);
```

인증에 성공하게 되면 현재의 스레드에 인증된 사용자의 정보가 설정되어 JEUS Security Manager에서 관리하는 자원에 대한 접근 제어를 수행할 수 있게 되며, 인증에 실패한 경우에는 guest 사용자로 접근한 것과 같이 구분되어 동작한다.

2.3 Context에서 Named Object 찾기

Java Client 응용 프로그램에서 서비스를 제공받기 위해서는 해당 객체를 찾아 와야 한다. 객체는 JEUS Namespace에 등록되어 JNDI Tree에 구성되어 있는 경우에 JNDI의 Context에서 `lookup()` 메소드를 통하여 찾을 수 있다.

다음의 코드에서는 `"ejbAppHome1"`이라고 이름 지어진 객체를 찾아오는 것을 보여주고 있다.

```
try {
    AppHome appHome1 = (AppHome) ctx.lookup("ejbAppHome1");
    // successfully got the object
} catch (NameNotFoundException ex) {
    // no such binding exists
} catch (NamingException e) {
    // an error occurred
}
```

2.4 Named Object에서 Object Reference 얻기

EJB Client 응용프로그램에서는 EJB Home에서 EJB Remote 객체에 대한

Reference를 가져오게 된다. 이와 같이 Client 응용프로그램에서 JEUS JNDI를 통해 얻어온 객체로부터 서비스를 받기 위한 Reference를 가져 올 때에는 해당 객체에서 Remote 객체의 Reference를 얻는 메소드를 호출하게 되며, 얻어온 Reference를 통해 Remote 객체에 대한 요청을 하게 된다.

```
AppBean bean = appHome1.create();
bean.service();
```

2.5 Context 종료하기

Context를 얻어 와서 모두 사용한 후에는 close() 메소드를 호출하여 해당 Context를 종료한다.

```
try {
    ctx.close();
} catch (Exception e) {
    // an error occurred
}
```

3 서버 측 객체에서 JEUS JNDI 사용하기

EJB Bean이나 Servlet, JSP와 같이 JEUS가 서비스하는 객체들은 JEUS의 각각의 엔진들과 같은 JVM에서 동작하게 된다. 또한 JEUS에서 이들을 위하여 JEUS JNDI를 사용할 수 있는 환경을 구성하여 두고 있다. 따라서 서버 측에서 동작하는 객체들을 일반적인 Java Client와 달리 JEUS JNDI를 사용하기 위해서 아무런 설정을 하지 않아야 한다.

서버 측 객체에서는 다음과 같은 과정으로 JEUS JNDI를 사용하게 된다.

- InitialContext 생성하기
- Context에서 Named Object 찾기
- Named Object에서 Object Reference를 얻어 사용하기
- Context 종료하기

다음에서는 이들의 각각의 과정에 대해서 자세히 설명하도록 하겠다.

3.1 InitialContext 생성하기

JEUS 엔진들과 동일한 JVM에서 생성되어 동작하는 객체가 Context를 얻을 때에는 아무런 설정을 하지 않아도 된다. 즉 Initial Context Factory를 위한 `java.naming.factory.initial` 또는 JNSServer에게 연결하기 위한 `java.naming.provider.url` 과 같은 변수를 설정하지 않고 다음과 같은 InitialContext의 생성자에 아무런 인자 없이 생성하게 된다. 기본 설정에 의해 Server-Side Object에서의 Naming Service는 로컬 노드의 JNSServer에 연결해서 동작하는 JNSLocal에서 제공받는다.

```
Context ctx = new InitialContext();
```

위와 같이 Server-Side Object에서 기본적인 설정 이외에 사용자의 정보를 설정하거나 JNDI Tree를 관리하기 위한 설정을 위해서는 해당 환경 변수를 추가할 수 있다.

3.2 Context에서 Named Object 찾기

특정한 객체로부터 서비스를 제공받기 위해서는 해당 객체를 찾아 와야 한다. 객체는 JEUS Namespace에 등록되어 JNDI Tree에 구성되어 있는 경우에 JNDI의 Context에서 `lookup()` 메소드를 통하여 찾을 수 있다.

다음의 코드에서는 "ejbAppHome1"이라고 이름 지어진 객체를 찾아오는 것을 보여주고 있다.

```
try {
    AppHome appHome1 = (AppHome) ctx.lookup("ejbAppHome1");
    // successfully got the object
} catch (NameNotFoundException ex) {
    // no such binding exists
} catch (NamingException e) {
    // an error occurred
}
```

3.3 Named Object에서 Object Reference 얻기

EJB Bean에서 다른 EJB를 호출하거나 Servlet이나 JSP에서 EJB 등을 사용하기 위해서는 JEUS JNDI를 통해서 EJB Home을 얻어 오게 되며, EJB Home에서 EJB Remote 객체에 대한 Reference를 가져오게 된다. 이와 같이 JEUS JNDI를 통해 얻어온 객체로부터 서비스를 받기 위한 Reference를 가져 올 때에는 해당 객체에서 Remote 객체의 Reference를 얻는 메소

드를 호출하게 되며, 얻어온 Reference
를 통해 Remote 객체에 대한 요청을 하게 된다.

```
AppBean bean = appHome1.create();  
bean.service();
```

3.4 Context 종료하기

Context를 얻어 와서 모두 사용한 후에는 close() 메소드를 호출하여 해당 Context를 종료한다.

```
try {  
    ctx.close();  
} catch (Exception e) {  
    // an error occurred  
}
```

4. 클러스터링 환경에서 JEUS JNDI 사용하기

JEUS JNDI는 J2EE 서비스를 위한 Naming Service를 제공하기 위해서 클러스터링 환경에서 동작할 수 있도록 설계되었다. EJB나 RMI, JMS와 같이 클러스터링 환경에서 서비스를 제공하는 객체들을 JNDI Tree에 등록하고 클라이언트에서 등록된 객체들을 찾아와 사용할 수 있도록 한다.

다음에서는 클러스터링 환경에서 JEUS JNDI를 통하여 서비스를 사용하고 필요한 동작을 위한 객체를 등록하는 여러 가지 방법에 대하여 설명하고 있다.

4.1 클러스터링 환경 구성하기

JEUS JNDI는 JEUS 시스템이 구성하고 있는 클러스터링 환경에서 동작하게 되며, JNSServer와 JNSLocal의 구성으로 클러스터링 환경을 구축하게 된다. 본 매뉴얼의 1장에서 설명한 것과 같이 각각의 노드에 하나씩 동작하게 되는 JNSServer는 클러스터링 환경에서 가장 중요한 역할을 수행하며, 전체 JNDI Tree를 구성하고 관리하는 역할을 한다.

또한 각각의 JEUS 엔진들과 함께 동작하는 JNSLocal은 자신의 로컬 노드에서 동작하고 있는 JNSServer에 연결하여 클러스터링 환경에서 구성되어

있는 JNDI Tree의 내용을 찾거나 새로운 객체를 등록하는 등의 역할을 수행하며 클라이언트에게 JEUS JNDI의 Naming Service를 제공하게 된다.

4.2 클러스터링 환경에 객체 등록하기

클러스터링 환경에서 전체의 JNDI Tree에 등록되어 있는 객체들은 클러스터 내의 모든 클라이언트에서 찾아 볼 수 있는 객체들이다. 이 객체들은 JNSLocal로부터 클라이언트가 등록하고자 하는 객체를 받아 각각의 JNSServer를 통하여 모든 노드에서 공유하게 되며 데이터의 삭제나 변경이 일어났을 때에는 전체 JNDI Tree에 해당 정보가 전파되어 각 노드의 JNSLocal에까지 반영된다.

JNDI Tree에 객체를 등록할 때에 해당 객체가 가지는 특성에 따라 다양한 옵션을 설정할 수 있다. 어떤 객체는 모든 JNDI Tree에서 공유하여 어느 노드에서나 찾을 수 있어야 하기 때문에 전체 클러스터링 환경에 전파 시켜야 할 필요가 있고, 또 한편 다른 객체는 자신의 노드 안에서만 찾을 수 있으면 되기도 하고, 때론 자신의 JVM 안에서만 찾을 수 있으면 되는 경우도 있다.

이러한 경우에 각각 적절한 객체의 등록을 위해서 다음과 같은 옵션을 조합하여 사용할 수 있으며, 이 설정을 통하여 효율적인 JNDI Tree를 구성할 수 있게 된다.

- JEUSContext.REPLICATE_BINDINGS
객체를 등록할 때에 모든 JNDI Tree에서 공유할 수 있도록 한다. 기본값은 true 이다.
- JEUSContext.CACHE_BINDINGS
객체를 찾아와 JNSLocal에서 Cache할 수 있도록 한다. 기본값은 true 이다.
- JEUSContext.LOCAL_BINDINGS
객체를 등록할 때에 자신의 JNSLocal에만 등록한다. 기본값은 false 이다.

이들은 서로 조합하여 사용할 수 있으며, 의미가 있는 조합은 다음과 같다.

환경 변수	설정 값 (True/False)
-------	-------------------

REPLICATE_BINDINGS	Don't Care	true	true	false	false
CACHE_BINDINGS	Don't Care	true	false	true	false
LOCAL_BINDINGS	true	false	false	false	false

표 2-1. 객체 등록 환경 변수의 조합

위와 같은 다섯 가지 경우에는 각각 다음과 같은 동작을 하게 된다.

- LOCAL_BINDINGS=true
객체를 자신의 JVM에만 등록한다.

(이하 LOCAL_BINDINGS=false인 경우)

- REPLICATE_BINDINGS=true / CACHE_BINDINGS=true
객체를 전체 JNDI Tree에 등록하고 Caching을 한다.
- REPLICATE_BINDINGS=true / CACHE_BINDINGS=false
객체를 전체 JNDI Tree에 등록하되 Caching을 하지 않는다.
- REPLICATE_BINDINGS=false / CACHE_BINDINGS=true
객체를 로컬 노드에만 등록하고 Caching을 한다.
- REPLICATE_BINDINGS=false / CACHE_BINDINGS=false
객체를 로컬 노드에만 등록하되 Caching을 하지 않는다.

이러한 환경 변수들을 설정하기 위해서는 JNDI의 InitialContext를 생성하기 위해서 환경 변수를 설정할 때에 함께 Hashtable에 등록하거나 Java 인터프리터의 실행 옵션으로 지정할 수 있으며, 또한 다음과 같이 JNDI의 Context에서 addToEnvironment() 메소드를 통하여 설정할 수 있다.

```
ctx.addToEnvironment(JEUSContext.REPLICATE_BINDINGS, "true");
ctx.addToEnvironment(JEUSContext.CACHE_BINDINGS, "true");
ctx.addToEnvironment(JEUSContext.LOCAL_BINDINGS, "false");
ctx.bind("name", object);
```

이들의 옵션에 대한 자세한 설명은 다음과 같다.

4.2.1 REPLICATE_BINDINGS

클러스터링 환경에서 전체 JNDI Tree에서 사용할 수 있도록 객체를 등록하기 위해서는 JNSLocal로부터 등록된 객체를 JNSServer에게 전달하여 각각의 노드에 존재하는 JNSServer들에게까지 전달해야 한다.

REPLICATE_BINDINGS 옵션을 true로 설정하게 되면 이와 같이 전체의 JNDI Tree에서 사용할 수 있도록 객체를 전체 JNSServer들에게 전파하게 되고, false로 설정하게 되면 로컬 노드에 있는 JNSServer에게만 전달하게 된다. 객체를 Replicate하는 구조를 다음 그림과 같이 살펴 볼 수 있다.

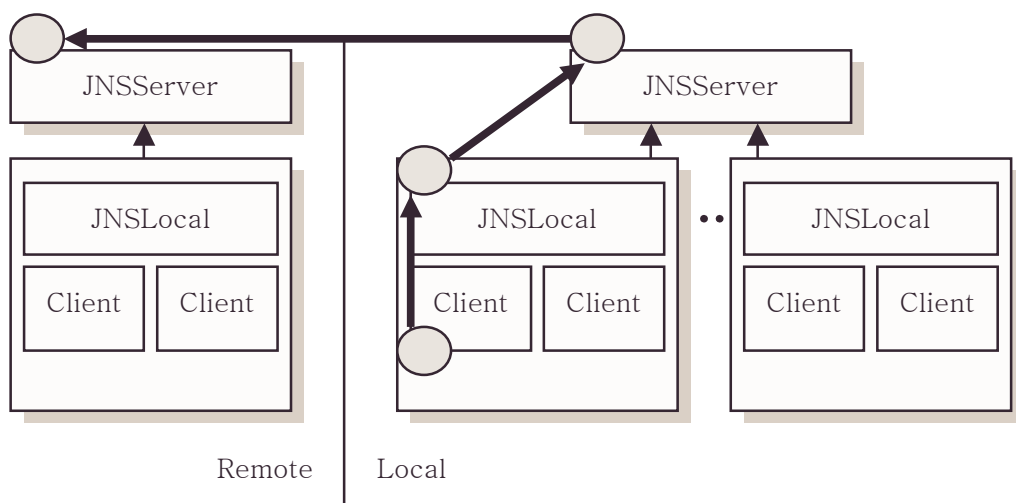


그림 2-1. Replicate Bindings

4.2.2 CACHE_BINDINGS

클라이언트가 JEUS JNDI를 통하여 객체를 찾아올 때에는 JNSLocal이 해당 객체를 자신이 연결한 JNSServer로 연결하여 객체를 찾아오며, CACHE_BINDINGS를 true로 설정하게 되면 자신의 저장 공간에 찾아온 객체를 Cache하게 된다.

JNSLocal에서 Cache하고 있는 객체를 다시 클라이언트에서 요청하게 되면 JNSServer에게 요청하지 않고 Cache하고 있던 객체를 돌려주게 된다. 다음 그림은 Cache되어 있는 객체를 돌려주는 관계를 나타낸 것이다.

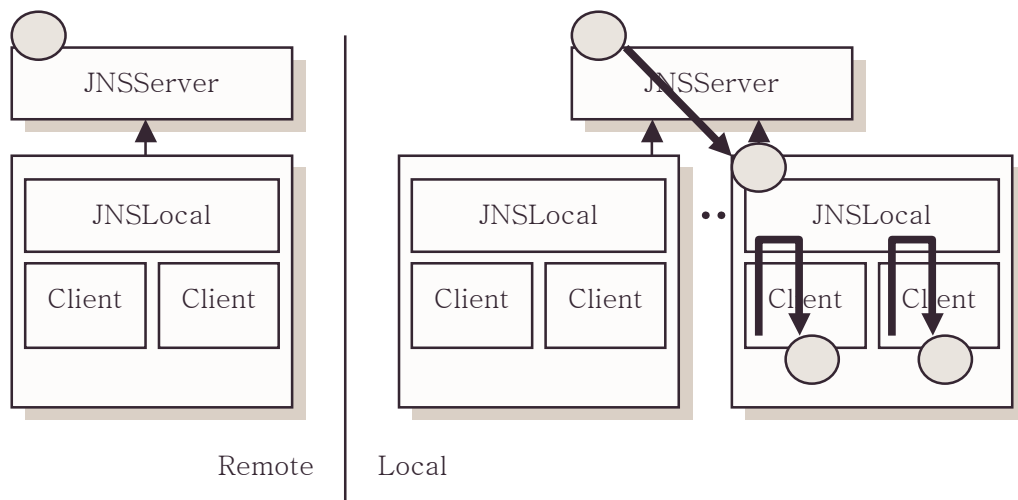


그림 2-2. Cache Bindings

4.2.3 LOCAL_BINDINGS

클러스터링 환경에서 구성하고 있는 전체의 JNDI Tree에서 모든 부분에서 보일 필요가 없이 일부에서만 찾아서 사용하는 객체들의 경우에는 LOCAL_BINDINGS 옵션을 통해서 효율적인 등록을 할 수 있다. 이 항목을 true로 설정하게 되면 JNSLocal에서만 등록이 되어 전체 JNDI Tree에서는 보이지 않도록 등록이 되며, 각각의 JVM 마다 클라이언트들의 요청을 JVM 안에서 처리할 수 있게 된다.

다음 그림에서는 JNSLocal에만 객체를 등록하여 JNSServer에게 요청하지 않고 객체를 찾아 주는 것을 보여 주고 있다.

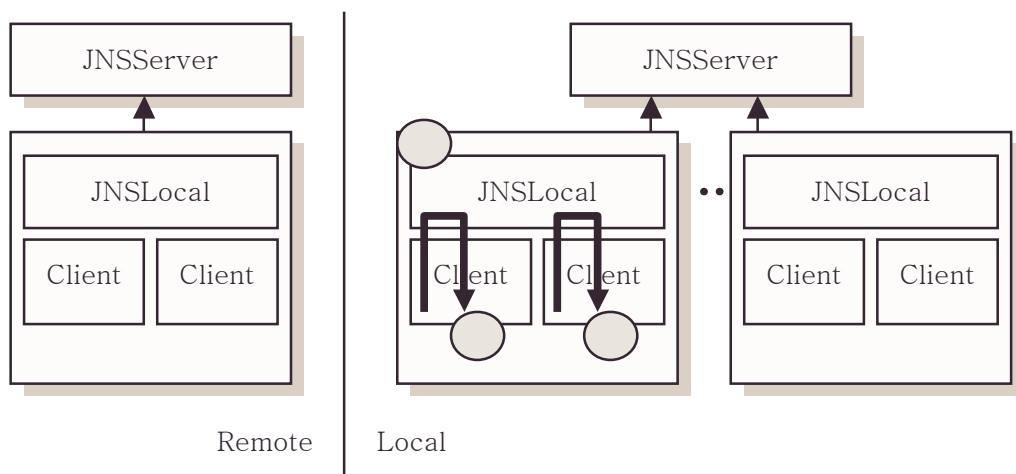


그림 2-3. Local Bindings

Ⅲ. JEUS Naming Server 설정하기

1. 개요

이 장에서는 JEUS Naming Server가 동작하는 데에 필요한 설정 사항에 대한 내용을 다루며 기본적인 동작에 필요한 사항과 클러스터링 환경에서 필요한 사항, 보안에 관한 설정을 통해 Naming Service를 관리하는 내용에 대하여 설명하도록 하겠다.

2. 기본 설정 사항

JEUS Naming Server는 JNSServer와 JNSLocal로 구성되어 동작하며 이들은 각각 독립적인 설정 사항을 가지고 있다. 기본적인 설정 사항에서 JNSServer는 JNSLocal로부터의 연결을 받아들이고 다른 JNSServer와의 연결을 관리하는 등에서 필요한 자원에 대한 설정을 하게 된다. 한편 JNSLocal의 경우에는 JNSServer에게 연결하고 JNDI Tree의 내용에 대한 반영을 하는 데에 필요한 설정을 하게 된다.

2.1 JNSServer 설정

JEUS Naming Server의 JNSServer의 설정은 JeusMain.xml 파일을 통해서 수행하게 된다. JEUS Manager가 동작하면서 JNSServer를 실행하게 되므로 관리자 툴을 이용하거나 XML 파일을 수정하여 JNSServer의 설정을 할 수 있다.

다음에 보여주고 있는 내용은 JeusMain.xml 파일에서 JNSServer에 관한 설정 부분을 발췌해서 나타낸 이다.

```
<JeusSystemConfig>
    . . . .
    <NamingServerConfig>
        <Server>
            <ExportCosNaming>true</ExportCosNaming>
            <Resolution>60000</Resolution>
            <ServerBufferSize>20480</ServerBufferSize>
            <ServerBackLogSize>50</ServerBackLogSize>
            <ThreadPool>
                <MinPoolSize>5</MinPoolSize>
                <MaxPoolSize>10</MaxPoolSize>
            </ThreadPool>
        </Server>
    </NamingServerConfig>
</JeusSystemConfig>
```

```

        <IncreasingStep>1</IncreasingStep>
        <ResizingPeriod>60000</ResizingPeriod>
    </ThreadPool>
</Server>
<Local>
    <Resolution>60000</Resolution>
    <LocalBufferSize>20480</LocalBufferSize>
    <ThreadPool>
        <MinPoolSize>3</MinPoolSize>
        <MaxPoolSize>10</MaxPoolSize>
        <IncreasingStep>1</IncreasingStep>
        <ResizingPeriod>60000</ResizingPeriod>
    </ThreadPool>
</Local>
</NamingServerConfig>
</JeusSystemConfig>

```

JNSServer의 기본 동작에 필요한 설정 사항으로는 다음과 같은 내용을 들 수 있다.

- ExportCosNaming

CosNaming Server를 띄울 것인지 설정하는 부분이다. 이것을 true로 설정하면 JEUS BASEPORT (시스템 프로퍼티 -Djeus.baseport)로부터 4만큼 증가한 (JEUS BASEPORT + 4) 포트에 tnameserver가 뜬다.

- Resolution

이 항목은 JNSServer에서 사용하는 자원에 대한 검사를 수행하는 주기를 설정하는 부분이며 1/1000초 단위로 설정할 수 있다. 기본값은 60000으로 설정되어 있으며, 따라서 기본값에 의해 60초마다 사용하고 있는 자원에 대한 검사를 하게 된다.

- ServerBufferSize

이 항목은 JNSLocal, JNSServer들과 통신 할 때에 사용하는 버퍼의 크기를 나타내며 byte 단위로 설정할 수 있다. 기본값은 20480 byte로 설정되어 있다.

- ServerBacklogSize

이 항목은 JNSLocal과 JNSServer로부터의 연결을 받아 들일 때의 Backlog의 한계를 지정하는 것이며 기본값으로 50으로 설정되어 있다.

- Thread Pool

이 항목에서는 서비스를 수행하기 위한 스레드 풀의 성질을 지정하는 부분이다. Naming Server에 가해지는 부하에 따라 풀의 크기를 조절하여 설정하여 안정된 시스템을 운영할 수 있다. 이 항목에는 다음과 같은 사항을 설정할 수 있다.

- MinPoolSize – 스레드 풀의 최소 크기를 설정할 수 있다.
- MaxPoolSize – 스레드 풀의 최대 크기를 설정할 수 있다. 스레드 풀이 최대 크기까지 커지게 되면 새로운 스레드를 실행하기 위해서 스레드 풀에 여유가 생길 때까지 기다리게 된다.
- IncreasingStep – 스레드 풀이 증가되는 개수의 단위를 설정한다. 현재 스레드 풀에서 더 이상 사용할 수 있는 스레드가 없을 경우에 이 항목에서 지정한 숫자 만큼 스레드 개수를 증가시키게 된다.
- ResizingPeriod – 사용하지 않는 스레드를 정리하는 주기를 1/1000초 단위로 설정할 수 있다. 이 항목에서 설정한 시간이 지나도록 사용하지 않고 있는 스레드들은 스레드 풀의 최소 크기까지 줄이게 된다.

위와 같은 항목을 설정하고 관리자 툴을 이용하여 변경하는 것에 대한 자세한 설명은 관리자 툴 매뉴얼을 참고하기 바란다.

2.2 JNSLocal 설정 (Server Side)

JEUS Naming Server의 Server Side JNSLocal 설정은 JeusMain.xml 파일을 통해서 수행하게 된다. JEUS Manager가 동작하면서 JNSLocal을 실행하게 되므로 관리자 툴을 이용하거나 XML 파일을 수정하여 JNSLocal의 설정을 할 수 있다.

다음에 보여주고 있는 내용은 JeusMain.xml 파일에서 Server Side JNSLocal에 관한 설정 부분을 발췌해서 나타낸 이다.

```
<JeusSystemConfig>
    . . .
    <NamingServerConfig>
        <Server>
            <ExportCosNaming>true</ExportCosNaming>
```

```

    <Resolution>60000</Resolution>
    <ServerBufferSize>20480</ServerBufferSize>
    <ServerBackLogSize>50</ServerBackLogSize>
    <ThreadPool>
        <MinPoolSize>5</MinPoolSize>
        <MaxPoolSize>10</MaxPoolSize>
        <IncreasingStep>1</IncreasingStep>
        <ResizingPeriod>60000</ResizingPeriod>
    </ThreadPool>
</Server>
<Local>
    <Resolution>60000</Resolution>
    <LocalBufferSize>20480</LocalBufferSize>
    <ThreadPool>
        <MinPoolSize>3</MinPoolSize>
        <MaxPoolSize>10</MaxPoolSize>
        <IncreasingStep>1</IncreasingStep>
        <ResizingPeriod>60000</ResizingPeriod>
    </ThreadPool>
</Local>
</NamingServerConfig>
</JeusSystemConfig>

```

■ Resolution

이 항목은 JNSLocal에서 사용하는 자원에 대한 검사를 수행하는 주기를 설정하는 부분이며 1/1000초 단위로 설정할 수 있다. 기본값은 60000으로 설정되어 있으며, 따라서 기본값에 의해 60초마다 사용하고 있는 자원에 대한 검사를 하게 된다.

■ LocalBufferSize

이 항목은 JNSServer와 통신 할 때에 사용하는 버퍼의 크기를 나타내며 byte 단위로 설정할 수 있다. 기본값은 20480 byte로 설정되어 있다.

■ Thread Pool

이 항목에서는 서비스를 수행하기 위한 스레드 풀의 성질을 지정하는 부분이다. JNSLocal에 가해지는 부하에 따라 풀의 크기를 조절하여 설정하여 안정된 시스템을 운영할 수 있다. 이 항목에는 다음과 같은 사항을 설정할 수 있다.

- MinPoolSize – 스레드 풀의 최소 크기를 설정할 수 있다.
- MaxPoolSize – 스레드 풀의 최대 크기를 설정할 수 있다. 스레드 풀이 최대 크기까지 커지게 되면 새로운 스레드를 실행하기 위해서 스레드 풀에 여유가 생길 때까지 기다리게 된다.

-
- IncreasingStep - 스레드 풀이 증가되는 개수의 단위를 설정한다. 현재 스레드 풀에서 더 이상 사용할 수 있는 스레드가 없을 경우에 이 항목에서 지정한 숫자 만큼 스레드 개수를 증가시키게 된다.
 - ResizingPeriod - 사용하지 않는 스레드를 정리하는 주기를 1/1000초 단위로 설정할 수 있다. 이 항목에서 설정한 시간이 지나도록 사용하지 않고 있는 스레드들은 스레드 풀의 최소 크기까지 줄이게 된다.

2.3 JNSLocal 설정 (Client Side)

Client Side JNSLocal이 동작할 때에는 JNSServer에게 연결하고 JNDI Tree의 변경을 반영하게 되는 스레드가 존재한다. 스레드 풀에 대한 설정과 함께 JNSLocal의 기본 설정에 대한 사항으로 다음과 같은 내용을 들 수 있다.

■ JEUSContext.RESOLUTION

JNSLocal가 동작하면서 사용하는 자원에 대한 검사와 스레드 풀을 조절하는 등의 작업을 수행하는 주기를 1/1000초 단위로 설정할 수 있다. 기본값은 30000으로 설정되어 있어 30초마다 자원에 대한 검사를 수행하게 된다.

■ JEUSContext.THREAD_POOLING

JNSLocal에서 JNSServer에게서 JNDI Tree의 변경 사항을 받아들이는 서비스 스레드 풀을 설정할 수 있다. 이 때에 설정할 수 있는 사항으로는 스레드 풀의 최소 크기, 최대 크기, 증가 단위, 크기 조절 주기의 네 가지 사항이 있으며 다음과 같은 형식으로 설정하게 된다. 기본값은 2:10:2:60000으로 되어 있다.

MinPoolSize:MaxPoolSize:IncreasingStep:ResizingPeriod

■ JEUSContext.CONNECT_TIMEOUT

JNSLocal에서 JNSServer로 연결을 시도할 때에 바로 연결을 성공하지 못했을 경우 연결이 성립될 때까지 기다리는 시간을 설정할 수 있다. 이 단위는 초 단위이며, 설정한 시간동안 연결을 시도한 후에 연

결이 실패하면 예외를 발생시킨다. 기본값은 60으로 되어 있다.

■ JEUSContext.CONNECTION_DURATION

이 항목은 JNSLocal에서 JNSServer에게 연결을 맺어 두는 시간을 설정하는 항목으로, 설정한 시간동안 JNDI를 사용하지 않게 되면 JNSServer에게서 연결을 끊게 된다. 이후에 새로운 요청을 할 때에는 자동적으로 새로운 연결을 맺어 정상적으로 요청을 처리하게 되며, 이 값을 -1로 설정하거나 아무런 설정을 하지 않게 되면 기본값으로 영속적인 연결을 유지하게 된다. 이 항목의 설정 값의 단위는 1/1000 초이며, RESOLUTION 항목에서 설정한 시간의 주기마다 현재 연결이 사용되고 있는 상태를 판단하여 연결을 끊거나 유지하게 된다.

위와 같은 설정 사항을 JNSLocal에게 적용하기 위해서는 Java 인터프리터를 실행할 때에 실행 옵션(-D)으로 항목을 정의하거나 InitialContext에 Hashtable로 전달하는 환경 변수의 종류로 항목을 설정할 수 있다. 이들 환경 변수에 대한 더욱 자세한 설명은 본 매뉴얼의 5장 환경 변수 목록을 참고하기 바란다.

Note : Server-Side Object (EJB Module, Servlet/JSP 모듈)의 경우에는 JEUS에서 설정하여 동작시킨 JNSLocal를 이용하게 되므로 추가적인 설정이 필요 없다.

3. 클러스터링 환경 설정

JEUS Naming Server는 클러스터링 환경에서 JNSServer와 JNSLocal의 구성을 통하여 전체 시스템에서 공유하게 되는 JNDI Tree를 구성하고 관리한다. 각각의 노드를 연결하기 위한 JNSServer의 설정과 여기에 연결하는 JNSLocal의 설정을 설명하겠다.

3.1 JNSServer 설정

JNSServer는 서로 다른 JNSServer들과 함께 전체 클러스터링에서 fully connect된 연결을 유지하게 된다. JNSServer는 JEUS Manager가 실행하게 되며 JeusMain.xml에서 설정한 노드에 대한 리스트를 가지고 전체 클러스

터링 환경에서 연결하게 될 JNSServer들을 찾아 각각의 JNSServer들이 동작하게 된다.

3.2 JNSLocal 설정 (Server Side)

Engine Container 내의 JNSLocal은 로컬노드의 JNSServer와 연결을 유지한다. 엔진 내의 모듈(EJB, Servlet/JSP 모듈등)은 이를 이용한다.

3.3 JNSLocal 설정 (Client Side)

JNSLocal는 JNSServer에게 연결을 맺게 되며, Context.PROVIDER_URL 항목을 통해 연결하게 될 JNSServer의 주소를 설정할 수 있다. 기본적으로는 로컬 노드로 연결한다.

```
ht.put(Context.PROVIDER_URL, "<hostname>");
```

4. 보안 설정

JEUS Naming Server를 동작시켜 Naming Service를 제공할 때에 사용자를 인증하고 Naming Service의 자원에 대한 접근 제어를 설정할 수 있다. JEUS JNDI에서는 기본적으로 JNDI API에서 정의되어 있는 방법을 통하여 사용자가 인증 과정을 수행할 수 있도록 하고 있으며, 인증된 사용자가 해당 자원을 사용할 수 있는지에 대한 접근 제어를 하게 된다.

JEUS Naming Server는 JEUS 시스템이 구성되어 동작할 때에 이와 함께 맞물려 동작하게 되므로 보안에 관련된 내용을 설정하기 위해서는 JEUS에서 제공하는 관리 툴과 설정 파일을 이용하게 된다. 다음에서는 JEUS Naming Server에서 사용자 인증과 접근 제어를 설정을 하는 과정에 대하여 설명하도록 하겠다.

한편 이러한 과정을 툴을 이용하여 수행하는 것에 대한 더욱 자세한 설명이 필요할 때에는 JEUS 관리자 툴 매뉴얼을 참고하기 바란다.

4.1 사용자 인증 설정

JEUS Naming Service를 이용할 수 있는 사용자를 등록하기 위해서는 JEUS에서 제공하는 File Realm에 사용자를 추가하여야 한다. 즉, JEUS Naming Server가 JEUS JNDI를 통하여 인증 정보를 입력한 사용자를 확인할 때에는 JEUS Security의 File Realm을 이용하는 것이다. 사용자를 등록하거나 정보를 변경하고 삭제하는 등의 기능을 위해서 JEUS Administration Tool을 이용하거나 Administration Console을 이용할 수 있다.

4.1.1 사용자 등록

사용자를 등록할 때에는 다음과 같은 방법들이 가능하다.

- JEUS Administration Tool을 이용한 사용자 등록

화면 좌측의 Tree에서 Security 항목을 펼친다.

User 항목을 오른쪽 마우스 버튼으로 선택한다.

나타난 Pop-up 창에서 Add 기능을 선택한다.

Add User Dialog에서 사용자 이름과 비밀번호를 입력한다.

User 항목을 펼쳐 추가된 사용자를 확인한다.

- JEUS Administration Console을 이용한 사용자 등록

JEUS의 system 역할을 가진 사용자로 로그인한다.

프롬프트에서 addUser (au) 기능을 수행한다.

listUser (lu) 기능으로 등록된 사용자를 확인한다.

4.1.2 사용자 수정 및 삭제

사용자의 정보를 수정하거나 삭제하는 과정은 유사하며, 다음과 같이 수행할 수 있다.

- JEUS Administration Tool을 이용한 사용자 수정 및 삭제

화면 좌측의 Tree에서 Security 항목을 펼친다.

User 항목을 펼쳐 수정하거나 삭제할 사용자를 오른쪽 버튼으로 선택

택한다.

나타난 Pop-up 창에서 목적에 맞게 Modify, 혹은 Delete 기능을 선택한다.

Modify 기능을 선택했을 때에는 User Modify Dialog에서 사용자의 정보를 수정한다.

User 항목을 펼쳐 사용자의 정보가 수정된 것을 확인한다.

■ JEUS Administration Console을 이용한 사용자 수정 및 삭제

JEUS의 system 역할을 가진 사용자로 로그인한다.

수정 시에는 password (pw), assignUserGroup (aug), assignUserRole (aur) 기능 중에서 필요한 기능을 수행한다.

삭제 시에는 deleteUser (du) 기능을 수행한다.

listUser (lu), infoUser (iu) 등의 기능으로 사용자의 정보를 확인한다.

4.2 접근 제어 설정

JEUS Naming Server에서 서비스하는 Context들 중에서 일부의 사용자들에게만 열람이나 수정 등의 기능을 허용할 수 있다. 이는 JEUS에서 제공하는 보안 기능 중에서 자원에 대한 접근 제어를 설정함으로써 이루어 지게 된다. 이에 JEUS Naming Service의 자원에 대한 접근 제어를 설정하는 것을 살펴보도록 하겠다.

4.2.1 자원 보호 설정

자원을 설정할 때에는 JEUS Naming Service에서 제공하는 서비스의 대상이 되는 Context를 단위로 하여 구성하게 된다. JEUS Namespace를 구성하고 있는 Context들 중에서 사용자가 접근하는 것에 대한 제어를 하고자 할 때에는 해당 Context의 이름과 기능을 가지고 자원을 설정하여 JEUS 보안 체계에서 이를 보호하도록 한다. JEUS Naming Service의 자원은 다음과 같은 구성을 가지게 되며, 이를 설정하기 위해서는 JEUS Administration Tool이나 Administration Console을 이용할 수 있다.

■ Resource Type = jndi

JEUS Naming Service에서의 자원은 자원의 형태로 “jndi”형태를 가진다. 한편 이 형태의 자원은 전체 JEUS 시스템에서 함께 사용되는 자원으로 인식된다.

■ Resource Name = <context path>

JEUS Naming Service에서의 자원의 이름은 보호하고자 하는 Context의 경로를 가진다. 이 경로는 JEUS Namespace에서 가지고 있는 절대 경로 값을 나타내며 ‘*’문자를 이용하여 해당 Context 하부의 모든 경로에 대한 자원으로서의 의미를 부여할 수 있다.

■ Resource Action = lookup | modify | list

JEUS Naming Service에서 자원에 대한 동작으로는 lookup, modify, list 의 세 가지 기능을 지정할 수 있다. 모든 동작을 하나의 자원으로 포함하기 위해서는 ‘*’ 문자로 나타낼 수 있다.

위와 같은 세 가지 정보, 즉 자원의 형태와 이름, 동작에 대한 값으로 자원을 구성하여 보호하게 되면 해당 자원을 사용하고자 하는 사용자에게 접근 제어를 수행하게 된다. 자원을 보호하도록 설정할 때에는 다음과 같은 방법을 통하여 Administration Tool이나 Administration Console에서 수행할 수 있다.

■ JEUS Administration Tool을 이용하여 자원 보호하기

- 화면 좌측의 Tree에서 Security 항목을 펼친다.
- ACL 항목을 마우스의 오른쪽 버튼으로 선택한다.
- 나타난 Pop-up 창에서 Add 기능을 선택한다.
- Add Resource Dialog에서 자원의 정보를 입력한다.
- ACL 항목을 펼쳐 자원이 추가된 것을 확인한다.

■ JEUS Administration Console을 이용하여 자원 보호하기

- JEUS의 system 역할을 가진 사용자로 로그인한다.
- 프롬프트에서 protect (pr) 기능을 수행한다.
- listACL (la) 기능으로 추가된 자원을 확인한다.

4.2.2 접근 허가 설정

JEUS 보안 체계에서 사용자가 보호되고 있는 자원에 대하여 접근하기 위해서는 접근이 허가된 역할을 부여 받아야만 한다. 즉 해당 자원에 대해서 A라는 역할이 접근 허가되어 있다면 A 역할을 가지고 있는 사용자 만이 접근할 수 있게 되는 것이다. 자원에 대한 접근을 특정 역할에게 허가하거나 불허하기 위해서 JEUS Administration Tool과 Administration Console에서는 다음과 같은 과정을 수행할 수 있다.

- JEUS Administration Tool에서 접근 허가 설정하기
 - 화면 좌측의 Tree에서 Security 항목을 펼친다.
 - ACL 항목을 펼쳐 설정하려는 자원을 마우스의 오른쪽 버튼으로 선택한다.
 - 나타난 Pop-up 창에서 Modify 기능을 선택한다.
 - ACL Modify Dialog에서 자원에 접근을 허가할 역할을 설정한다. 좌측의 역할 목록에서 허가하고자 하는 역할을 선택하여 우측의 Granted Role List로 이동시킨다.
 - 자원 항목을 선택하여 나타난 상세 정보 창에서 자원에 허가된 역할을 확인한다.
- JEUS Administration Console에서 접근 허가 설정하기
 - JEUS의 system 역할을 가진 사용자로 로그인한다.
 - 프롬프트에서 grant (gr) 기능으로 허가하거나 deny (dr) 기능으로 불허한다.
 - infoACL (ia) 기능으로 자원에 허가된 역할을 확인한다.

4.2.3 자원 보호 해제

더 이상 자원에 대한 보호를 하지 않고 해제할 때에는 해당 자원에 대해 허가했던 역할들과의 관계를 모두 해지 시키고 난 후에 자원 보호를 해제해야 한다. JEUS Administration Tool이나 Administration Console을 이용

하여 다음과 같은 과정을 통해서 자원에 대한 허가를 해지하고 자원의 보호를 해제할 수 있다.

■ JEUS Administration Tool을 이용한 자원 보호 해제

- 화면 좌측의 Tree에서 Security 항목을 펼친다.
- ACL 항목을 펼쳐 해제하려는 자원을 마우스의 오른쪽 버튼으로 선택한다.
- 나타난 Pop-up 창에서 Modify 기능을 선택한다.
- ACL Modify Dialog에서 자원에 접근이 허가된 역할을 모두 선택하여 허가를 해지한다.
- Tree에서 자원을 마우스 오른쪽 버튼으로 선택하여 Delete 기능을 선택한다.
- ACL 항목을 펼쳐 자원의 보호가 해제 된 것을 확인한다.

■ JEUS Administration Console을 이용한 자원 보호 해제

- JEUS의 system 역할을 가진 사용자로 로그인한다.
- 프롬프트에서 infoACL (ia) 기능을 통해 자원에게 접근이 허가된 역할의 목록을 확인한다.
- deny (dr) 기능으로 보호를 해제하고자 하는 자원에 허가된 모든 역할에 대한 허가를 해지한다.
- 이어서 unprotect (ur) 기능으로 자원 보호를 해제한다.
- listACL (la) 기능으로 보호되고 있는 자원의 목록에서 해당 자원이 제거된 것을 확인한다.

IV. JEUS Directory Service 이용하기

1. 개요

이 장에서는 JEUS JNDI를 사용하여 Java 응용 프로그램을 작성하거나 클러스터링 환경에서 JEUS JNDI를 사용하여 Directory Service를 이용하는 방법을 설명하도록 하겠다. 크게 특정 Object에 관련된 Attribute을 수정, 조회하는 Attribute Service와 NameSpace와 Object의 변화를 통지 받을 수 있는 Event Service 두 종류가 있다. JNDI Specification에 나와 있는 표준적인 JNDI API를 이용하여 이 서비스를 이용하면 된다. 좀 더 자세한 예제와 정보는 JNDI 관련 Tutorial을 참조하라.

2. Attribute Service

JEUS는 Attribute Service를 제공하여 객체와 연관된 속성(Attribute)을 찾고 수정하는 기능을 제공한다. 이를 이용하려면 아래의 패키지를 import하여야 한다.

```
import javax.naming.directory.*;
```

JEUS에서 제공하는 Attribute Service를 이용하는 예를 예제와 함께 설명 하겠다.

2.1 Attribute 등록

아래는 특정 객체와 객체 emp의 속성을 “jeus”라는 이름으로 Namespace에 Bind하는 예제이다.

```
DirContext initial = new InitialDirContext();
Employee emp = new Employee();
Attributes empAttrs = new BasicAttributes();
empAttrs.put("name", "jeus");
empAttrs.put("age", "32");
empAttrs.put(new BasicAttribute("sex", "male"));
initial.rebind("jeus", emp, empAttrs);
```

2.2 Query

아래는 Namespace에서 'name' 속성이 존재하고 'age' 속성이 20 인 객체들을 찾는 예제이다.

```
DirContext initial = new InitialDirContext();
// ignore case
Attributes matchAttrs = new BasicAttributes(true);
matchAttrs.put(new BasicAttribute("age", "20"));
matchAttrs.put(new BasicAttribute("name"));

NamingEnumeration ne = initial.search("", matchAttrs);
```

2.2 Query – 검색 필터(SearchFilter) 이용하기

검색을 할 때, Attribute의 조건을 Logical Expression 형태로 스트링으로 표현할 수 있다. 아래는 검색 필터(Search Filter)를 이용한 예이다.

```
DirContext initial = new InitialDirContext();
String filter = "(&(age=20)(name=*))";
NamingEnumeration ne = initial.search("", filter, null);
```

나이가 20 보다 많고 이름은 어떤 것도 상관없는 조건을 나타내는 경우 필터 스트링을 나타내려면 아래와 같이 한다.

(&(age=20)(name=*))

다음은 검색 필터에 사용할 수 있는 기호들의 간단한 리스트이다.

```
&   : AND
|   : OR
!   : NOT
=   : 같음
~=  : 비슷함
>=  : 크거나 같음
<=  : 작거나 같음
=*  : 존재함
*   : 와일드 카드
\   : 특수문자를 사용할 경우 Escape Character
```

2.2 Query – 검색 제어(SearchControl) 이용하기

검색 필터 이외에도 다양한 검색 제어를 주어서 검색할 수 있는 데, SearchControl이 검색 제어를 설정하는 용도로 사용된다. 아래

SearchControl을 주고 검색하는 예제를 보였다.

```
DirContext initial = new InitialDirContext();
String filter = "(&(age=20)(name=*))";
SearchControls cons = new SearchControls(SearchControls.SUBTREE_SCOPE, 10,
    10, new String[] {"name"}, true, true);
NamingEnumeration ne = initial.search("", filter, cons);
```

SearchControl의 생성자에 아래와 같이 다양한 제어를 설정할 수 있다.

```
public SearchControls(int scope,
    long countlim,
    int timelim,
    java.lang.String[] attrs,
    boolean retobj,
    boolean deref)
```

- scope
검색범위를 말한다. 3가지 중 하나를 설정할 수 있다.
OBJECT_SCOPE - 해당 Object만 검색
ONELEVEL_SCOPE - 해당 Context의 하위 레벨까지 검색
SUBTREE_SCOPE - 해당 Context의 SUBTREE를 모두 검색
- timelim
결과가 나오기 까지의 1/1000초 단위의 검색 제한 시간. 만약 0
이라면 영원히 기다림.
- deref
검색중 Link를 Dereference할 것인지 설정
- countlim
결과로 나올 엔트리의 제한. 만약 0이라면 모든 엔트리를 리턴함.
- retobj
객체를 반환할 것인지 설정. True라면 객체 리턴. False면 객체를
리턴하지 않음.
- attrs
결과로 리턴할 Attribute 리스트. 만약 null이라면 모든 Attribute
를 리턴함.

2.2 NamingEnumeration

검색의 결과는 NamingEnumeration 타입으로 넘어온다. NamingEnumeration을 사용하여 다음과 같이 목록에 저장되어 있는 이름과 속성을 콘솔에 출력할 수 있다.

```
NamingEnumeration ne = . . . .;
while (ne.hasMore()) {
    SearchResult sr = (SearchResult)ne.next();
    // print its name
    System.out.println("Name: " + sr.getName());
    Attributes attrs = sr.getAttributes();
    for (NamingEnumeration ae = attrs.getAll(); ae != null &&
        ae.hasMoreElements();) {
        Attribute attr = (Attribute)ae.next();
        System.out.println("    attribute: " + attr.getID());
        // print each value
        for (NamingEnumeration e = attr.getAll(); e.hasMoreElements();)
            System.out.println("        value: " + e.nextElement());
    }
}
```

3. Event Service

JEUS는 NameSpace의 변화를 통지 받을 수 있고 Object의 변화를 통지 받을 수 있는 Event Service를 제공한다. 이를 이용하려면 아래의 패키지를 import하여야 한다.

```
import javax.naming.directory.*;
import javax.naming.directory.event.*;
```

JEUS에서 제공하는 Event Service를 이용하는 예를 예제와 함께 설명하겠다.

3.1 NamespaceChangeListener

Namespace에 변화가 일어났을 경우 이를 통지 받을 수 있는 서비스를 말한다. Namespace 변화에는 객체의 추가, 삭제 그리고 이름의 변화가 있다. 아래 Namespace 변화시 이를 콘솔에 출력하는 간단한 예제가 있다.

```
class NCListener implements NamespaceChangeListener {
    public void objectAdded(NamingEvent evt) {
        System.out.println("Object Added: " + evt.getNewBinding());
    }
    public void objectRemoved(NamingEvent evt) {
        System.out.println("Object Removed: " + evt.getOldBinding());
    }
}
```

```

    }

    public void objectRenamed(NamingEvent evt) {
        System.out.println(id + "Object Renamed: " + evt.getNewBinding() +
            " from " + evt.getOldBinding());
    }

    public void namingExceptionThrown(NamingExceptionEvent evt) {
        evt.getException().printStackTrace();
    }
}

```

3.2 ObjectChangeListener

Namespace내의 특정 객체에 변화가 일어났을 경우 이를 통지 받을 수 있는 서비스를 말한다. 객체에 변화가 있을 경우 이전에 Binding된 객체와 새로이 바인딩된 객체를 NamingEvent에 넣어서 이를 알려준다.

```

class OCLListener implements ObjectChangeListener {
    public void objectChanged(NamingEvent evt) {
        System.out.println("Object Changed: " + evt.getNewBinding() + "
            from " + evt.getOldBinding());
    }
    public void namingExceptionThrown(NamingExceptionEvent evt) {
        evt.getException().printStackTrace();
    }
}

```

3.3 Listener 등록하기

위에서 구현한 NamespaceChangeListener와 ObjectChangeListener를 네임서버에 등록하는 예를 아래에서 보았다.

```

InitialDirContext initial = new InitialDirContext();
EventDirContext ctx = (EventDirContext) initial.lookup("somenam");
ctx.addNamingListener("listenername", EventDirContext.SUBTREE_SCOPE, new
    NCLListener("NCLListener"));

```

V. 환경 변수 목록

1. 개요

이 장에서는 JEUS JNDI에서 설정할 수 있는 주요 환경 변수의 목록을 정리하도록 하겠다. 기본적인 JNDI의 `javax.naming.Context` 인터페이스에 정의되어 있는 환경 변수와 JEUS JNDI에서만 특별하게 `jeus.jndi.JEUSContext` 인터페이스에 정의되어 있는 환경 변수를 사용할 수 있다.

이들을 사용하려면 다음과 같이 import를 한다.

```
import javax.naming.*;  
import jeus.jndi.*;
```

2. JEUS JNDI 환경 변수

환경 변수	내용
Context.INITIAL_CONTEXT_FACTORY	Initial Context를 생성하기 위한 설정으로, JEUS JNDI를 이용하기 위해서는 <code>jeus.jndi.JEUSContextFactory</code> 로 설정한다.
Context.PROVIDER_URL	JNSLocal에서 연결하게 될 JNSServer의 호스트 주소를 설정한다. 기본값은 <code>127.0.0.1</code> 이다.
Context.URL_PKG_PREFIXES	JEUS JNDI에서 Java URL Scheme을 이용하기 위해서 설정하는 항목이며 <code>jeus.jndi.jns.url</code> 로 설정한다.

환경 변수 (Continued...)	내용
Context.SECURITY_PRINCIPAL	인증 과정을 위해서 설정하는 사용자의 Principal 항목이며 기본값은 guest 로 되어 있다.
Context.SECURITY_CREDENTIALS	인증 과정에서 필요한 사용자의 비밀번호를 설정하는 항목이다.
JEUSContext.RESOLUTION (jeus.jndi.jns.resolution)	JNSLocal이 동작하면서 사용하는 자원을 관리하는 주기를 설정할 수 있다. 1/1000초 단위로 설정할 수 있으며 기본값은 30000으로 되어 있다.
JEUSContext.THREAD_POOLING (jeus.jndi.jns.threadpooling)	JNSLocal이 동작하면서 사용하는 스레드 풀을 설정할 수 있다. 스레드 풀의 최소 크기, 최대 크기, 증가 단위, 크기 조절 주기를 설정할 수 있으며 기본 값은 2:10:2:60000 으로 되어 있다.
JEUSContext.CONNECT_TIMEOUT (jeus.jndi.jns.connecttimeout)	JNSLocal이 JNSServer에게 연결을 시도하는 시간을 설정할 수 있다. 설정한 시간만큼 연결을 시도하게 되며, 시간이 지나도록 연결에 실패하면 예외를 발생시킨다. 기본값은 60이며, 설정 단위는 초 단위이다.
JEUSContext.CONNECTION_DURATION (jeus.jndi.jns.connectionduration)	JNSLocal에서 JNSServer로의 연결을 유지하는 시간을 설정할 수 있다. 1/1000초 단위로 설정이 가능하며 설정한 시간이 지나도록 JNDI를 사용하지 않으면 연결을 끊게 된다. 기본값은 -1로 되어 있으며, 이것은 연속적인 연결을 유지한다는 것을 의미한다.

환경 변수 (Continued...)	내용
JEUSContext.REPLICATE_BINDINGS (jeus.jndi.jns.replicatebindings)	등록하고자 하는 객체를 JNDI Tree 전체에 전파 시키게 된다. 기본 값은 true이며, false로 설정하게 되면 로컬 노드에 있는 JNSServer에게만 등록된다.
JEUSContext.CACHE_BINDINGS (jeus.jndi.jns.cachebindings)	JNDI Tree에서 찾아온 객체를 해당 Depend- ent Server에서 Caching 하게 된다. 기본 값은 true이며, false로 설정하게 되면 객체를 찾을 때마다 JNSServer에게 요청하게 된다.
JEUSContext.LOCAL_BINDINGS (jeus.jndi.jns.localbindings)	JNDI Tree 중 해당 JNSLocal에에만 유효한 객체를 등록하게 된다. 기본 값은 false이며, true로 설정하게 되면 다른 REPLICATE_BINDINGS, CACHE_BINDINGS 옵션이 무시된다.

표 4-1. JEUS JNDI 환경 변수 목록